Google Cloud H2S

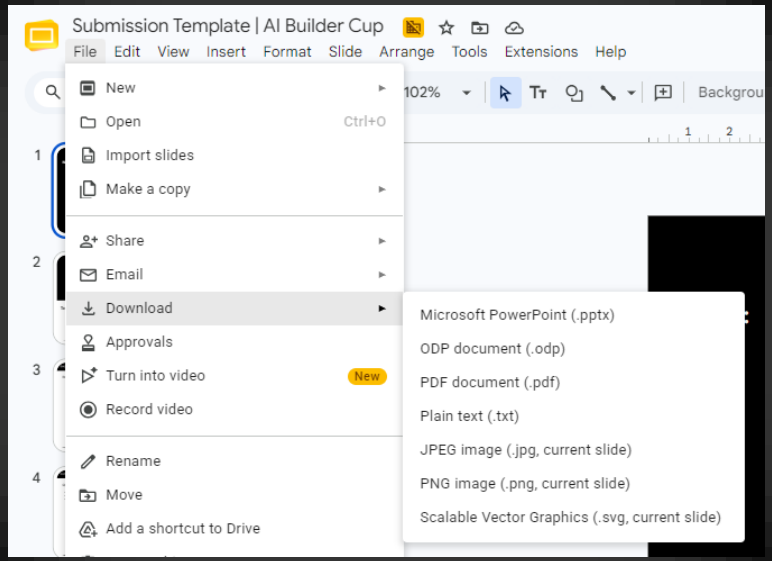
AI Builder Cup

Build in JAPAC.
Build for the world.



Team Details

a. Team name: gTaprobane
b. Team leader name: Yogendran Nadaraja
c. Problem Statement: A bank's headline percent rarely covers the whole basket; the real saving is the basket total after each offer's exclusions, minimum bill, and cap.


Submission Template | AI Builder Cup
File
Edit
View
Insert
Format
Slide
Arrange
Tools
Extensions
Help
New
Open
Ctrl+O
Import slides
Make a copy
Share
Email
Download
Approvals
Turn into video
New
Record video
Rename
Move
Add a shortcut to Drive
Microsoft PowerPoint (.pptx)
ODP document (.odp)
PDF document (.pdf)
Plain text (.txt)
JPEG image (.jpg, current slide)
PNG image (.png, current slide)
Scalable Vector Graphics (.svg, current slide)

Brief about the idea

Savvy Cart AI is a condition-aware grocery basket optimizer for Sri Lankan households. It reads a shopper's receipts with Gemini, works out what they are running low on, and plans the cheapest **legal** way to buy it on a chosen day — across up to four stores, one card per bill — applying every exclusion, minimum bill, cap, and store or card rule in deterministic code, not guessing with a model.

Its **Offer Reality Check** lets a shopper paste a bank offer and see what the headline promises against what the terms actually pay on their own basket. The app is live on Cloud Run with Gemini and the Agent Development Kit — a guest, no-login, five-step flow that ends in a MOCK approval (no order is sent).

Opportunities

- How different is it from any of the other existing ideas?
- How will it be able to solve the problem?
- USP of the proposed solution

• **How different:** price-comparison apps show the cheapest store per item. Savvy Cart prices the full basket after card-offer exclusions, minimums, caps, and cannot-combine rules — and shows, line by line, when a famous “25% off” actually pays LKR 0.

• **How it solves the problem:** Gemini turns receipts and offer text into structured data; an exact OR-Tools optimizer assigns items to stores and applies every rule; the shopper sees baseline vs optimized and the best day, then approves. Nothing is auto-ordered.

• **USP:** the AI reads documents; deterministic code owns every rupee, so the plan can be trusted. Two grounding layers stop the model inventing a price, discount, or total.

List of features offered by the solution

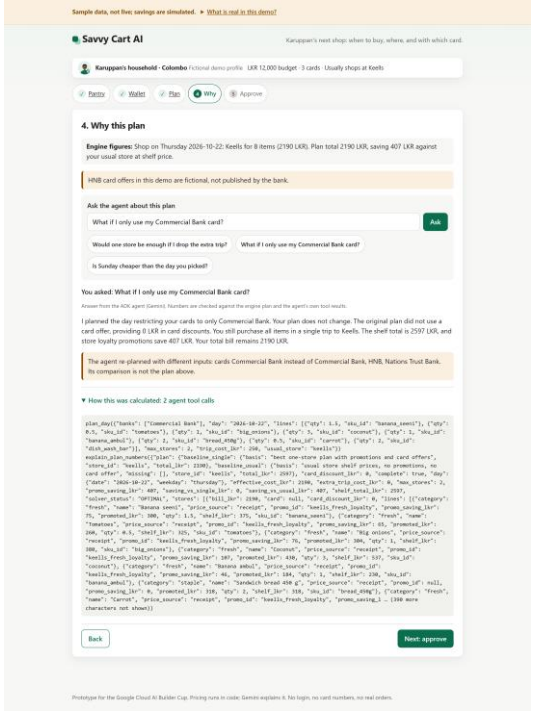
- Receipt-driven pantry: due dates from real purchase history (median gap) — no forecasting model.
- Gemini receipt extraction: a photo becomes catalog-matched lines with a confidence per line.
- Offer Reality Check: Gemini reads a bank offer’s text or screenshot into validated terms.
- Exact multi-store optimizer (OR-Tools CP-SAT): up to four stores, store promotions, one card offer per bill, trip-cost aware.
- Ask the agent (ADK): natural-language questions, on-demand re-planning, and a visible tool trace.
- Honest by design: “this offer refuses your basket” (LKR 0), no invented numbers, MOCK approval, and no card numbers ever.

Process flow diagram or Use-case diagram

Guest opens the Cloud Run URL — no login — and moves through a five-step wizard:

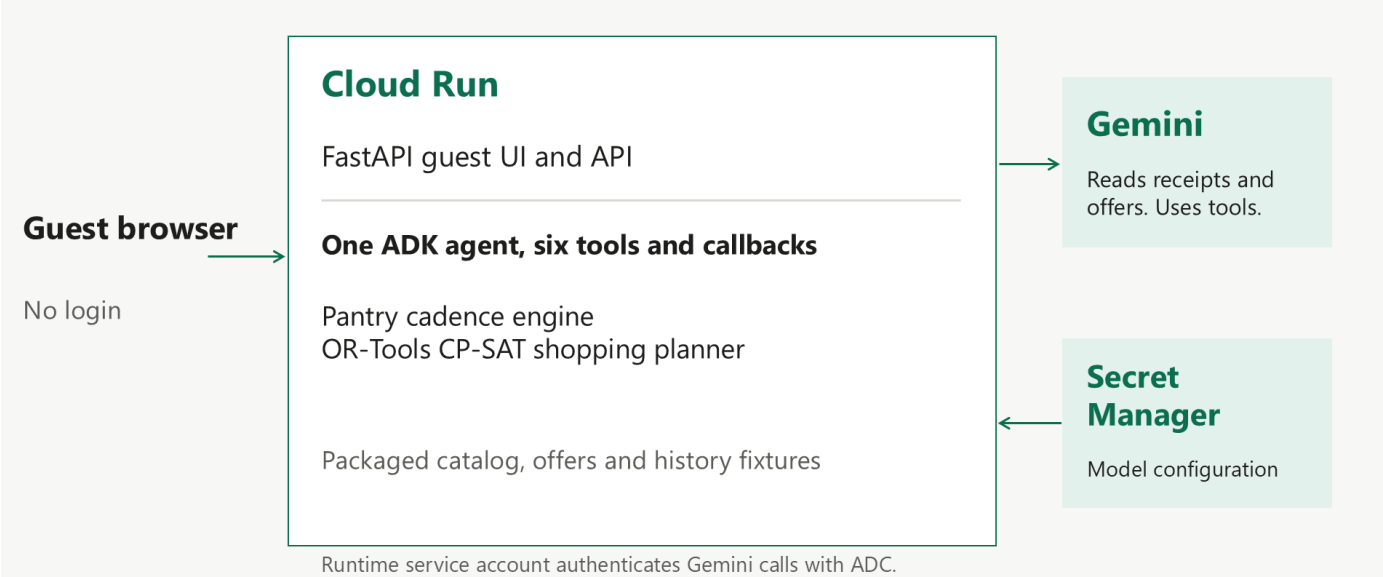
- **1 Pantry —** receipt history shows what is running low; add a receipt (sample, or a photo read by Gemini).
- **2 Wallet —** choose cards, stores, and trip cost; run Offer Reality Check on a bank offer.
- **3 Plan —** the OR-Tools optimizer picks stores, store promotions, the best card offer, and the best day.
- **4 Why —** the ADK agent explains the plan, can re-plan, and shows every tool call it made.
- **5 Approve —** a MOCK order preview. No order is sent.

Wireframes/Mock diagrams of the proposed solution (optional)



The deployed guest UI is the working prototype. Shown: the **Why** step, where the ADK agent answers the shopper's question in natural language and shows every tool call it made (plan_day, explain_plan_numbers). Every number is checked against the engine before it is displayed.

Architecture diagram of the proposed solution



Cloud Run (FastAPI, scale-to-zero) hosts the guest UI, the one ADK agent, and the OR-Tools engine in a single container. Gemini (gemini-3.8-flash on the Agent Platform) reads receipts and offers and explains plans. Secret Manager holds the model id; the runtime uses keyless Application Default Credentials. Data is packaged fixtures.

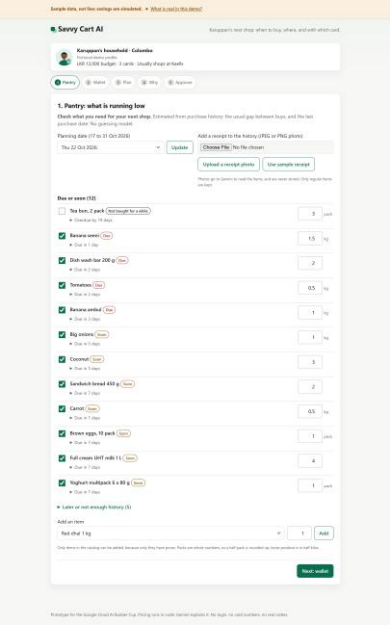
Technologies to be used in the solution

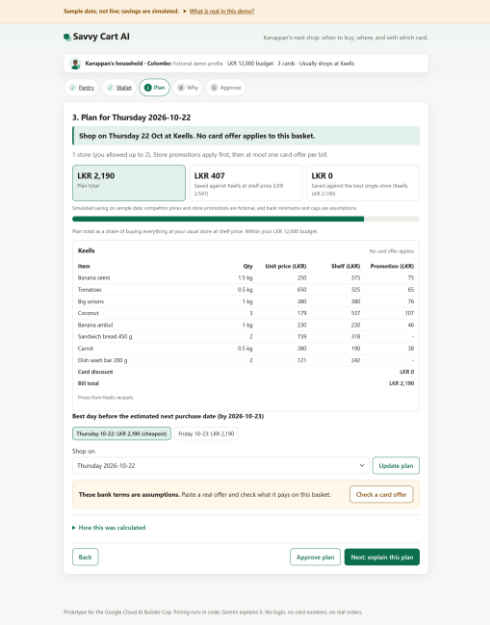
- **Google Cloud:** Cloud Run (asia-southeast1, scale to zero), Gemini gemini-3.8-flash on the Gemini Enterprise Agent Platform, Secret Manager.
- **GenAI & agents:** Agent Development Kit (google-adk 2.x) and google-genai.
- **Optimization:** Google OR-Tools (CP-SAT) exact solver.
- **Application:** Python 3.12, FastAPI, uvicorn; server-rendered HTML with a small vanilla JS/CSS frontend; httpx.
- **Build & test:** Docker (python:3.12-slim, non-root); pytest — 120 passing.
- **Not used in the prototype:** Firebase and Firestore — data is committed fixtures; Firestore is the documented path for production scale.

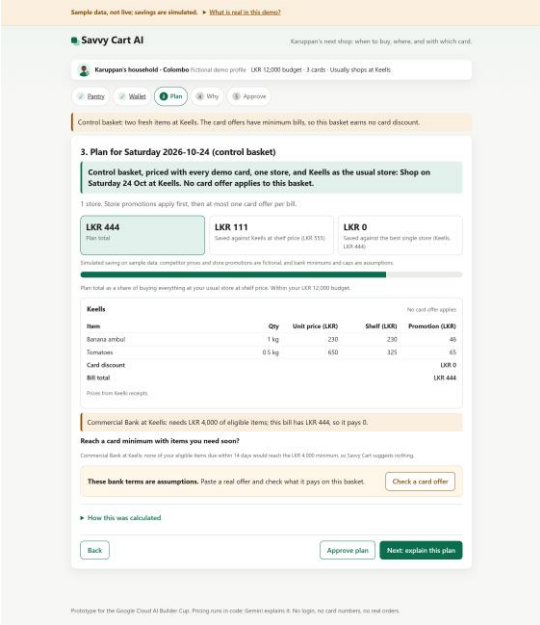
Estimated implementation cost (optional)

- Cloud Run scales to zero (min instances 0, max 3): no paid warm instances.
- Gemini Flash traffic is judge-only during evaluation.
- No managed database cost — the prototype runs on committed fixtures.
- Estimated spend over the evaluation window: single-digit to low-tens of USD, typically covered by trial credits.

Snapshots of the prototype







Live screens: the receipt-driven pantry; the optimized plan line by line; and the honest case where a card offer's minimum is not met and it pays LKR 0. Sample data — savings are simulated.

Prototype Performance report/Benchmarking

- 120 automated tests pass — deterministic pricing, offer eligibility, and guardrails.
- Live Gemini (18 Oct 2026): the Why answer about 30 seconds; reading a bank offer about 7 seconds.
- Worked example (simulated): a default basket costs LKR 4,604 across Keells and Cargills — LKR 1,328 below buying everything at Keells shelf price; the two-store split beats the best single store by LKR 63 (at a LKR 320 trip cost, one store wins).
- Offer Reality Check: a “10% off Saturday” headline that suggests LKR 593 pays LKR 0 — the basket is LKR 2,475 under the LKR 8,000 minimum.
- All savings are simulated on labeled sample data.

Additional Details/Future Development (if any)

- **Roadmap:** a Colombo household pilot, then authorized retailer/issuer data feeds (versioned prices, real card terms), then adapting the rules to other JAPAC markets (currencies, rounding, local card offers).
- **Business model:** a free household tool; later, issuers or retailers pay for consented, disclosed referrals — never selling the shopper's data.
- **Beyond the prototype:** live prices, real orders and payments, Firestore persistence, and partner checkout are all post-Cup.

Provide links to your:

- GitHub Public Repository https://github.com/nyogendran/savvy-cart-ai
- Demo Video Link (3 Minutes) (to be added before submission)
- Final Product Link https://savvy-cart-890425261137.asia-southeast1.run.app

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

